

**VERTOFI**

PREDICTIVE ACCOUNTING PLATFORM

OPERATIONS HANDBOOK

API Keys, Credentials & Environment Setup

Every external service, secret, and environment variable Vertofi needs — what it is, how to obtain it, exactly where to place it, and how to verify it works. Built for a production deployment on Google Cloud (GKE Autopilot) + Vercel.

GKE Autopilot · asia-south1

Vercel · 8 frontends

GCP Secret Manager

15+ providers

Source of truth: `.env.example` · `.env.production.example`

Rev. 2026-06-06 · Confidential

Contents

0	How configuration flows — the three placement targets, naming, philosophy, tools
1	Core infrastructure — GCP · Cloud SQL · Memorystore · GCS · Confluent · Secret Manager
2	Secrets you generate yourself — JWT · APP_FIELD_KEY · break-glass OTP
3	External provider credentials — 14 providers, one card each
4	Frontend (Vercel) variables — NEXT_PUBLIC_* + CORS
5	Placement mechanics — the exact commands for each target
6	Master matrix & minimum-to-boot — every variable at a glance
7	Verification playbook — curl + kubectl checks per service
8	Rotation & incident response
9	FAQ & troubleshooting

Legend used throughout:

HARD-REQUIRED

platform won't serve without it
remove after launch

REQUIRED*

needed for login / charging

GENERATED

you create it

OPTIONAL

degrades gracefully

TEMPORARY

0

FOUNDATIONS

How configuration flows

Vertofi reads **all** runtime config from environment variables — nothing with real values is committed. The whole game is knowing *where each variable lives*. There are exactly three places.

0.1 · The three placement targets

Target	Used by	How it gets there	When
A — Local <code>.env</code>	services on your machine (<code>pnpm dev</code> , docker-compose)	edit <code>E:\VERTOFI APPLICATION\.env</code> (from <code>.env.example</code>)	local dev
B — GCP Secret Manager	the 32 backend services on GKE	<code>sync-secrets.ps1</code> pushes each as <code>vertofi-shared-<KEY></code> ; External Secrets Operator projects them into pods	prod backend
C — Vercel env	the 8 Next.js frontends (<code>NEXT_PUBLIC_*</code>)	Vercel dashboard or <code>vercel env add</code>	prod frontend

RULE OF THUMB

Anything a **backend** service reads → **A** locally, **B** in prod. Anything starting `NEXT_PUBLIC_` is baked into the browser bundle → **C**. **Never put a real secret behind a `NEXT_PUBLIC_` name** — it ships to every visitor.

0.2 · Naming convention in Secret Manager

Every backend secret uses a shared prefix: `OPENAI_API_KEY` → Secret Manager secret `vertofi-shared-OPENAI_API_KEY` . ESO finds them by that prefix and injects the bare name into the pod. **One shared set is used by all services** — no per-service secrets.

0.3 · The "NEEDS_CREDENTIALS" philosophy

A missing credential **never crashes the platform and never fakes data**. Leave `AA_CLIENT_ID` blank → the bank connector returns `NEEDS_CREDENTIALS` and the UI shows an honest empty state. So you can launch with just the core + a minimal set and add the rest later.

HARD-REQUIRED

DB · Redis · Kafka · S3 · JWT × 2 ·
APP_FIELD_KEY · CORS_ORIGINS

TO LOG IN

SMTP or MSG91 — or temporary break-glass OTP

EVERYTHING ELSE

degrades gracefully to an empty state

0.4 · Tools you must install

- **Google Cloud SDK** (`gcloud`) — cloud.google.com/sdk/docs/install
- **kubectl** · **helm** · **terraform** (already present)
- **Vercel CLI** — `npm i -g vercel`
- **Node 22 + pnpm 10**, PowerShell for the `infra/gke/*.ps1` scripts

0.5 · Security rules (non-negotiable)

- 1 Never commit `.env` / `.env.production` with real values (they're gitignored — keep it so).
- 2 Never put a secret behind a `NEXT_PUBLIC_` name.
- 3 Use a dedicated mailbox / sub-account per provider so you can rotate without breaking personal logins.
- 4 Use test/sandbox keys until verified end-to-end, then swap to live (Razorpay, GST, AA especially).

- 5 Turn **off** `OTP_BYPASS_ENABLED` the moment real OTP delivery works.
- 6 Rotate any key that ever appears in a log, screenshot, or chat (Part 8).

1

REQUIRED

Core infrastructure

Mostly produced by `terraform apply`, but you must capture each connection string / key into `.env.production`.

1.1 · GCP project & billing **HARD**

- 1 Create/choose a project at console.cloud.google.com/projectcreate — note the **Project ID** (e.g. `vertofi-prod-001`).
- 2 Link a billing account: console.cloud.google.com/billing.
- 3 Authenticate: `gcloud auth login` · `gcloud config set project <id>` · `gcloud auth application-default login`.
- 4 Enable APIs:

```
gcloud services enable container.googleapis.com sqladmin.googleapis.com \
redis.googleapis.com storage.googleapis.com secretmanager.googleapis.com \
artifactregistry.googleapis.com vision.googleapis.com cloudbilling.googleapis.com \
iam.googleapis.com --project vertofi-prod-001
```

```
GOOGLE_CLOUD_PROJECT=vertofi-prod-001
GCP_REGION=asia-south1
GCP_BILLING_ACCOUNT_ID=XXXXXX-XXXXXX-XXXXXX
```

1.2 · Cloud SQL (Postgres 16) → DATABASE_URL **HARD**

Created by Terraform (HA, PITR, encrypted). Capture the private IP + password; use the Cloud SQL Auth Proxy for migrations.

```
DATABASE_URL=postgresql://vertofi:<PASS>@<PRIVATE_IP>:5432/vertofi
DB_SSL=true
DATABASE_URL_R0=postgresql://vertofi:<PASS>@<REPLICA_IP>:5432/vertofi # optional
```

BEFORE SIGNUPS

Run `pnpm migrate` per service against Cloud SQL (via the Auth Proxy or a one-shot Job).

1.3 · Memorystore → REDIS_URL **HARD**

Console → [memorystore/redis/instances](#). Powers sessions, AI-usage metering, rate limits, caches.

```
REDIS_URL=redis://<HOST>:6379
```

1.6 · Artifact Registry

Images at `asia-south1-docker.pkg.dev/<proj>/vertofi`, built by GitHub Actions. Repo secrets:

```
GCP_PROJECT_ID GCP_REGION GCP_SA_KEY
```

1.4 · Cloud Storage + HMAC → S3_* **HARD**

Vertofi uses the GCS **S3-compatible** API, so it needs S3-style HMAC keys for the runtime SA.

```
gcloud storage hmac create vertofi-runtime@vertofi-prod-001.iam.gserviceaccount.com \
--project vertofi-prod-001 # → Access ID (S3_ACCESS_KEY) + Secret (S3_SECRET_KEY)
```

```
S3_ENDPOINT=https://storage.googleapis.com S3_FORCE_PATH_STYLE=true
S3_REGION=asia-south1 S3_BUCKET=<documents bucket>
S3_ACCESS_KEY=<HMAC Access ID> S3_SECRET_KEY=<HMAC Secret>
```

1.5 · Confluent Cloud (Kafka) → KAFKA_* HARD

The entire event pipeline (document → OCR → categorization → ledger → BHS, audit black box) runs on Kafka.

- 1 Sign up at confluent.cloud; create a Basic cluster near asia-south1.
- 2 Cluster → **API keys** → create (cluster-scoped). Save Key + Secret.
- 3 Copy the **Bootstrap server** (`pkc-xxxxx.<region>.gcp.confluent.cloud:9092`).

```
KAFKA_BROKERS=pkc-xxxxx.asia-south1.gcp.confluent.cloud:9092
KAFKA_CLIENT_ID=vertofi KAFKA_SSL=true KAFKA_SASL_MECHANISM=plain
KAFKA_SASL_USERNAME=<Confluent API Key> KAFKA_SASL_PASSWORD=<Confluent API Secret>
```

PYTHON SERVICES

ocr · ai-gateway · categorization read the same SASL vars and build `security_protocol=SASL_SSL` automatically.

1.7 · Secret Manager + External Secrets Operator

Target **B**. You don't create secrets by hand — `sync-secrets.ps1` does it from `.env.production` ; ESO + the `ClusterSecretStore` `gcp-secret-manager` are installed by `deploy-gke.ps1` (commands in Part 5).

2

YOU CREATE THESE

Self-generated secrets

2.1 · JWT secrets

GENERATED

HARD

Used by `auth` to sign access + refresh tokens. **Services fail-fast in production if missing or <16 chars.** Use two different long random strings.

```
# PowerShell
[Convert]::ToBase64String((1..48 | %{Get-Random -Max 256})) # JWT_ACCESS_SECRET
[Convert]::ToBase64String((1..48 | %{Get-Random -Max 256})) # JWT_REFRESH_SECRET
# or: openssl rand -base64 48
```

```
JWT_ACCESS_SECRET=<random ≥16, unique> JWT_REFRESH_SECRET=<random, different>
JWT_ACCESS_TTL=900 JWT_REFRESH_TTL=2592000
```

2.2 · APP_FIELD_KEY (pgcrypto)

GENERATED

HARD

Symmetric key for at-rest field encryption in admin-console. Generate once and **never lose it** — losing it makes encrypted fields unreadable.

```
openssl rand -base64 32 # → APP_FIELD_KEY
```

2.3 · Break-glass OTP

TEMPORARY

Lets the pilot team log in *before* SMTP/SMS is live. Not a hardcoded code — you set a per-deployment operator secret.

```
OTP_BYPASS_ENABLED=1 OTP_BYPASS_CODE=<your 6-digit operator secret>
```

DISABLE AFTER LAUNCH

Set `OTP_BYPASS_ENABLED=0` (or remove both) the moment `SMTP_*` or `MSG91_*` deliver real codes. Leaving it on in production is a standing backdoor.

3

14 PROVIDERS · ONE CARD EACH

External provider credentials

OpenAI

OPENAI_API_KEY · AI_USD_PER_1K_TOKENS

STRONGLY RECOMMENDED

Powers ai-gateway → categorization · accounting AI-draft · Voice-CFO · legal analysis · WhatsApp AI. Without it the gateway degrades honestly (rules fallback; no AI drafts).

- 1 Sign up: platform.openai.com/signup; add billing + a usage cap.
- 2 Create a key (inside a Project for scoped spend): platform.openai.com/api-keys → copy `sk-...` (shown once).

```
OPENAI_API_KEY=sk-proj-xxxxxxx AI_USD_PER_1K_TOKENS=0.0006
```

PLACE

Backend (A/B)

VERIFY

ai-gateway /health → openai: configured;
create an AI-draft invoice

COST

Per-token; set a monthly cap

Google Cloud Vision

GOOGLE_VISION_KEY

RECOMMENDED

Powers the `ocr` service. Without it, uploads route to human review (`NEEDS_REVIEW`) instead of auto-extraction.

Option A — Workload Identity (recommended on GKE)

Leave the key blank; grant the runtime SA Vision access:

```
gcloud services enable vision.googleapis.com --project vertofi-prod-001
gcloud projects add-iam-policy-binding vertofi-prod-001 \
  --member serviceAccount:vertofi-runtime@vertofi-prod-001.iam.gserviceaccount.com \
  --role roles/serviceusage.serviceUsageConsumer
```

Option B — API key

console.cloud.google.com/apis/credentials → Create API key → restrict to *Cloud Vision API*.

```
GOOGLE_VISION_KEY=AIza...
```

VERIFY

Upload a clear invoice → fields populate

COST

1,000 units/mo free, then per-1,000 —
cloud.google.com/vision/pricing

Gemini

GEMINI_API_KEY

OPTIONAL

Optional secondary LLM. Leave blank if OpenAI-only (the default policy).

Key from Google AI Studio: aistudio.google.com/app/apikey → `GEMINI_API_KEY=AIza...` · Place: backend.

MSG91 — SMS OTP (India / TRAI-DLT)

MSG91_AUTH_KEY · MSG91_SENDER_ID · MSG91_TEMPLATE_ID

[LOGIN*](#)

Powers **auth** SMS OTP. Sending to Indian numbers legally requires **TRAI DLT** registration (sender ID + template approval). Budget a few business days.

- 1 Account: msg91.com/signup.
- 2 **DLT registration** (one-time): register on a DLT portal → get Principal Entity ID, an approved 6-char Header/Sender ID (e.g. **VERTFI**), and approved content templates (each gets a DLT Template ID).
- 3 Link DLT in MSG91, create + approve the OTP template.
- 4 Auth Key: control.msg91.com → Settings/API → Auth Key.

```
MSG91_AUTH_KEY=<auth key> MSG91_SENDER_ID=VERTFI MSG91_TEMPLATE_ID=<DLT template id>
# optional per-purpose: MSG91_TEMPLATE_ID_LOGIN / _REGISTER / _RESET / _FINANCIAL / _DOCUMENT / _MFA
```

PLACE

Backend

VERIFY

Request OTP with an Indian mobile →
SMS in seconds

COST

Per-SMS prepaid wallet

Email / SMTP

SMTP_HOST · _PORT · _USER · _PASS · _FROM

[LOGIN*](#)

Powers **auth** email OTP + **notification** dispatch. Any SMTP provider works.

Option A — Google Workspace app password (fastest)

- 1 Enable 2-Step Verification on the sending mailbox.
- 2 Create an App Password: myaccount.google.com/apppasswords → Mail → copy 16 chars.

```
SMTP_HOST=smtp.gmail.com SMTP_PORT=587 SMTP_USER=no-reply@vertofi.com
SMTP_PASS=<16-char app password> SMTP_FROM=no-reply@vertofi.com
```

Option B — Dedicated ESP (better at scale)

- **SendGrid:** host smtp.sendgrid.net , user literally [apikey](#) , pass = API key.
- **Brevo:** smtp-relay.brevo.com :587. **Mailgun:** smtp.mailgun.org .

DELIVERABILITY

Add **SPF** + **DKIM** + **DMARC** DNS records for vertofi.com or OTP emails land in spam. Each provider shows the exact records.

Meta WhatsApp Business

WHATSAPP_API_TOKEN · _PHONE_ID · _VERIFY_TOKEN · _APP_SECRET

OPTIONAL

Powers the `whatsapp` service (inbound webhook, AI replies, photo→document, plan-aware menus).

- 1 `developers.facebook.com` → Create App → Business; add the WhatsApp product.
- 2 Add your business number + complete Business Verification (`business.facebook.com`).
- 3 Generate a **permanent System-User token** with `whatsapp_business_messaging` + `_management`.
- 4 Note the **Phone Number ID**; get the **App Secret** (App → Settings → Basic).
- 5 Choose your own **verify token** string.
- 6 Webhook callback = `https://<whatsapp ingress>/webhook` (its own ingress, *not* the gateway); subscribe to `messages`.

```
WHATSAPP_API_TOKEN=<system-user token>  WHATSAPP_PHONE_ID=<phone id>
WHATSAPP_VERIFY_TOKEN=<your string>     WHATSAPP_APP_SECRET=<app secret>
```

KNOWN GAP

`wa_user` → `org` mapping is pending — replies/menus work; full per-org actions are limited until that table exists.

Razorpay — payments & subscriptions

RAZORPAY_KEY_ID · _KEY_SECRET · _WEBHOOK_SECRET · _PLAN_STARTER/GROWTH/PRO

TO CHARGE

Powers the `billing` service + `/subscribe` (7-day trial, UPI/card autopay). Without it billing degrades to "trial active".

- 1 Sign up + KYC at `dashboard.razorpay.com` (Test mode first).
- 2 Settings → **API Keys** → Generate → copy Key ID + Secret.
- 3 Subscriptions → **Plans** → one plan per tier → copy each `plan_id`.
- 4 Settings → **Webhooks** → URL `https://api.vertofi.com/api/v1/billing/webhook/razorpay`, set a secret, subscribe to `subscription.authenticated/charged/halted/pending/cancelled` + `payment.captured`.

```
RAZORPAY_KEY_ID=rzp_live_xxxx  RAZORPAY_KEY_SECRET=<secret>  RAZORPAY_WEBHOOK_SECRET=<yours>
RAZORPAY_PLAN_STARTER=plan_xxxx  RAZORPAY_PLAN_GROWTH=plan_xxxx  RAZORPAY_PLAN_PRO=plan_xxxx
```

VERIFY

Test mandate on `/subscribe` flips billing → Celebration screen

PLACE

Backend (webhook path is public in the gateway)

GST GSP

GST_GSP_BASE_URL · _CLIENT_ID · _CLIENT_SECRET

OPTIONAL

gst-connector. Blank → offline structural GSTIN validation only.

Pick a GSP (e.g. **Masters India** mastersindia.co/gst-api), onboard, get client creds + base URL (sandbox first).

```
GST_GSP_BASE_URL=https://api.mastersindia.co
GST_GSP_CLIENT_ID=... GST_GSP_CLIENT_SECRET=...
```

Account Aggregator (Setu)

AA_BASE_URL · _CLIENT_ID · _CLIENT_SECRET

OPTIONAL

bank-connector (RBI AA framework). Blank → empty state.

Use an AA gateway (e.g. **Setu** setu.co/data/account-aggregator); onboard for sandbox then prod creds.

```
AA_BASE_URL=https://fiu-sandbox.setu.co
AA_CLIENT_ID=... AA_CLIENT_SECRET=...
```

Accounting-sync targets OPTIONAL · **Verification connectors** OPTIONAL

Zoho Books — api-console.zoho.in → Server-based client → [ZHO_CLIENT_ID/SECRET](#) .

QuickBooks — developer.intuit.com → app keys → [QBO_CLIENT_ID/SECRET](#) .

Tally — [TALLY_CONNECTOR_URL=http://<tally-host>:9000](#) .

verification-connectors (stubs): [PAYROLL_API_KEY](#) , [CREDIT_BUREAU_API_KEY](#) , [MCA_API_KEY](#) . Blank → [NEEDS_CREDENTIALS](#) .

Admin-console extras: [GITHUB_TOKEN](#) + [GITHUB_REPO](#) (Actions tab, fine-grained PAT, Actions:Read) and [NEXT_PUBLIC_GRAFANA_URL](#) (Vercel, web-admin).

4

TARGET C

Frontend (Vercel) variables

Set per Vercel project (Settings → Environment Variables, Production). These are build-time `NEXT_PUBLIC_*` — public by design, never secrets. **Redeploy** after changing them.

Variable	Value	Projects
NEXT_PUBLIC_API_URL	https://api.vertofi.com/api/v1	all 8
NEXT_PUBLIC_BUSINESS_URL	https://business.vertofi.com	web-landing
NEXT_PUBLIC_ASSOCIATES_URL	https://associates.vertofi.com	web-landing
NEXT_PUBLIC_ACCOUNTANTS_URL	https://accountants.vertofi.com	web-landing
NEXT_PUBLIC_BHS_URL	https://bhs.vertofi.com	web-landing
NEXT_PUBLIC_LEGAL_URL	https://legal.vertofi.com	web-landing
NEXT_PUBLIC_GRAFANA_URL	https://grafana.... (optional)	web-admin

CORS MUST MATCH

The backend `CORS_ORIGINS` must list every domain above or the browser gets CORS errors.

```
CORS_ORIGINS=https://vertofi.com,https://www.vertofi.com,https://business.vertofi.com,
https://associates.vertofi.com,https://accountants.vertofi.com,https://bhs.vertofi.com,
https://legal.vertofi.com,https://admin.vertofi.com,https://teams.vertofi.com
RATE_LIMIT_PER_MIN=600
```

5

THE EXACT COMMANDS

Placement mechanics

5.1 · Local development (A)

```
Copy-Item .env.example .env      # fill DB/Redis/Kafka (docker), JWT secrets, OTP bypass
pnpm infra:up                   # Postgres/Redis/Redpanda/MinIO/OpenSearch
pnpm dev
```

5.2 · Production backend (B) → Secret Manager

```
cd infra\gke
.\sync-secrets.ps1 -EnvFile ../../.env.production -ProjectId vertofi-prod-001
# single value by hand:
gcloud secrets create vertofi-shared-OPENAI_API_KEY --replication-policy=automatic --project vertofi-prod-001
"sk-proj-xxxx" | gcloud secrets versions add vertofi-shared-OPENAI_API_KEY --data-file=- --project vertofi-prod-001
# apply now:
kubectl rollout restart deployment --all -n vertofi-identity # repeat per namespace
```

5.3 · Production frontend (C) → Vercel

```
vercel env add NEXT_PUBLIC_API_URL production # per project, see Part 4
vercel redeploy --target production
# after adding a Vercel domain, update CORS_ORIGINS (5.2) then:
kubectl rollout restart deployment api-gateway -n vertofi-gateway
```

6

AT A GLANCE

Master matrix & minimum-to-boot

Variable(s)	Source	Target	Required	Powers
DATABASE_URL · DB_SSL	Cloud SQL	B	HARD	everything
REDIS_URL	Memorystore	B	HARD	sessions/cache/metering
KAFKA_* (5)	Confluent	B	HARD	event pipeline
S3_* (6)	GCS + HMAC	B	HARD	document storage
JWT_ACCESS/REFRESH_SECRET (+TTL)	generated	B	HARD	auth tokens
APP_FIELD_KEY	generated	B	HARD	field encryption
CORS_ORIGINS · RATE_LIMIT_PER_MIN	—	B	HARD	gateway
OTP_BYPASS_ENABLED/CODE	generated	B	TEMP	break-glass login
SMTP_* (5)	Workspace/ESP	B	LOGIN*	email OTP + notifications
MSG91_* (3)	MSG91	B	LOGIN*	SMS OTP
OPENAI_API_KEY (+rate)	OpenAI	B	REC.	all AI
GOOGLE_VISION_KEY	GCP / WI	B	REC.	OCR
RAZORPAY_* (6)	Razorpay	B	CHARGE	billing/subscriptions
WHATSAPP_* · GST_GSP_* · AA_* · ZOHO/QBO/TALLY · PAYROLL/CREDIT/MCA · GEMINI · GITHUB_*	respective	B	OPTIONAL	degrade to empty state
NEXT_PUBLIC_API_URL (+ landing URLs)	—	C	HARD	frontends → gateway

* need at least one of SMTP / MSG91 (or break-glass) to log in.

ABSOLUTE MINIMUM TO BOOT & LOG IN (PILOT)

1 — Core infra: DATABASE_URL, DB_SSL, REDIS_URL, all KAFKA_*, all S3_*. 2 — Generated: JWT × 2, APP_FIELD_KEY. 3 — Gateway: CORS_ORIGINS. 4 — Login: SMTP_* or MSG91_* (or OTP_BYPASS). 5 — Frontend: NEXT_PUBLIC_API_URL (+ landing URLs). Everything else is incremental.

7

PROVE IT WORKS

Verification playbook

```
# 1 gateway up + public health
curl -s https://api.vertofi.com/health # {"status":"ok"}
# 2 login endpoint (public path)
curl -s -X POST https://api.vertofi.com/api/v1/auth/otp/send \
  -H "Content-Type: application/json" \
  -d '{"channel":"EMAIL","destination":"you@example.com","purpose":"LOGIN"}' # 202 {challengeId}
# 3 CORS for a frontend origin
curl -s -o /dev/null -w "%{http_code}\n" -X OPTIONS https://api.vertofi.com/api/v1/auth/otp/send \
  -H "Origin: https://business.vertofi.com" -H "Access-Control-Request-Method: POST" # 204 + ACAO
# 4 protected route needs a token
curl -s -o /dev/null -w "%{http_code}\n" https://api.vertofi.com/api/v1/accounting/x # 401
```

IN THE APPS

OTP arrives · invoice upload extracts (Vision) · "Ask Vertofi" drafts (OpenAI) · /subscribe test mandate flips billing · WhatsApp replies.

POD-LEVEL

`kubectl get pods -A` all Running · `kubectl logs deploy/auth -n vertofi-identity` no "missing env" · `kubectl get externalsecret -A` SecretSynced=True.

8

KEEP IT SAFE

Rotation & incident response

- **Rotate**: add a new Secret Manager version → restart consumers (`kubect1 rollout restart deployment --all -n <ns>`).
- **JWT rotation** invalidates all sessions (everyone re-logs in) — do it in a window; rotate access + refresh together.
- **Provider keys** (Razorpay/GST/AA/WhatsApp): revoke the old key in the dashboard after the new one is live.
- **Leak** (log/screenshot/chat): revoke at the provider immediately, add a new version, restart.
- **APP_FIELD_KEY** rotation requires re-encrypting existing encrypted columns — plan carefully.
- First hardening step post-launch: disable break-glass (`OTP_BYPASS_ENABLED=0`).
- Track owner + expiry per key (OpenAI project keys, Meta tokens, GSP creds expire).

9

QUICK ANSWERS

FAQ & troubleshooting

Set a secret but the service still says missing?

ESO hasn't synced or the pod wasn't restarted. Check `kubectl get externalsecret -A` (SecretSynced=True) and rollout-restart the service.

Browser shows a CORS error?

The origin isn't in `CORS_ORIGINS`. Add it, re-run sync-secrets, restart api-gateway.

Login OTP never arrives?

Neither SMTP nor MSG91 is working. Email → check DKIM/SPF + app password. SMS → DLT template/sender must be approved. Stopgap: break-glass.

Razorpay webhook not updating billing?

URL must be the public path `/api/v1/billing/webhook/razorpay` and `RAZORPAY_WEBHOOK_SECRET` must match the dashboard exactly.

Documents always go to "review"?

`GOOGLE_VISION_KEY` blank and Workload Identity not granted Vision. Configure one path (\$3.2).

Run with no AI key?

Yes — categorization uses deterministic rules, AI-draft is off; platform still works.

Where do `NEXT_PUBLIC_*` go?

Vercel (C). Public, build-time. Never a real secret behind that prefix.

End of guide. Keep this file internal. Update it whenever a new connector or variable lands in `.env.example` / `.env.production.example`.